

Protótipo para detecção, localização e seleção automática de ação por sensoramento

Requisitos Funcionais e Não Funcionais

Gabriel Vasconcellos de Moraes — Matheus Emanuel Gomes Souza

Orientador: Anker Douglas Loss

Curso: Engenharia da Computação — Faculdade do Centro Leste (UCL)

1 Apresentação

O RadarTorres é um protótipo que detecta alvos por sensoramento, rastreia-os em um radar circular, seleciona automaticamente a torre mais adequada e permite o acionamento manual ou automático de um indicador demonstrativo. Este documento reúne os requisitos funcionais (o que o sistema deve fazer, do ponto de vista de um ator) e os requisitos não funcionais (que características de qualidade essa funcionalidade deve ter) levantados a partir do código-fonte (`src/RadarTorres.App/`), do firmware (`Arduino/ArduinoSimulation.ino`) e da documentação técnica já existente no projeto.

Convenções: a **Prioridade** de um requisito funcional segue o critério de criticidade para a operação central do sistema (comunicação serial, rastreamento, seleção de torre, modos de operação e segurança recebem *Alta*; suporte administrativo e ferramental recebem *Média*; conveniências de UX recebem *Baixa*). O **Status** reflete o estado real de implementação hoje — *Implementado*, *Parcial* ou *Planejado* —, não a intenção original. Requisitos que descreviam decisão de implementação ou característica de qualidade em vez de comportamento observável foram removidos da lista funcional e reclassificados como não funcionais (RF07, RF25, RF32 → ver nota após a Tabela 1); a rastreabilidade completa de cada item até classe/método do código está em `Matriz_de_Rastreabilidade.md` (nesta mesma pasta), para não duplicar aqui.

2 Requisitos Funcionais

Tabela 1: Requisitos funcionais do RadarTorres

ID	Requisito	Ator(es)	Prior.	Status
RF01	Comunicação serial com o Arduino	Usuário, Arduino	Alta	Implementado
RF02	Recepção e validação de leituras de alvo	Arduino, Sistema	Alta	Implementado
RF03	Rastreamento de alvos em tempo real	Sistema	Alta	Implementado
RF04	Exibição do radar circular em tempo real	Usuário	Alta	Implementado
RF05	Seleção automática de torre	Sistema	Alta	Implementado
RF06	Acionamento demonstrativo automático	Sistema	Alta	Implementado (D1)
RF08	Modos de operação (Verde/Amarelo/Vermelho)	Usuário	Alta	Parcial (D1)
RF09	Autenticação multiusuário	Usuário	Alta	Implementado

Tabela 1 (continuação)

ID	Requisito	Ator(es)	Prior.	Status
RF10	Controle de acesso por perfil	Sistema, Usuário	Alta	Implementado
RF11	Troca de senha	Usuário	Média	Implementado
RF12	Registro histórico de detecções	Sistema	Média	Implementado
RF13	Visualização de objetos detectados	Usuário	Média	Implementado
RF14	Exportação de objetos detectados (CSV/XML/PDF)	Usuário	Baixa	Implementado
RF15	Importação de objetos detectados (CSV/XML)	Usuário	Baixa	Implementado
RF16	Auditoria de ações realizadas	Sistema, Usuário	Média	Parcial
RF17	Auditoria de alterações de modo	Sistema, Usuário	Média	Parcial
RF18	Gerenciamento de usuários	Usuário (Admin)	Média	Planejado
RF19	Preferências de usuário (idioma, tema, sidebar)	Usuário	Baixa	Implementado
RF20	Troca de idioma da interface	Usuário	Baixa	Implementado
RF21	Troca de tema visual	Usuário	Baixa	Implementado
RF22	Abertura de chamado de ajuda	Usuário	Baixa	Implementado
RF23	Tratamento administrativo de chamados	Usuário (Admin)	Baixa	Planejado
RF24	Painel principal com cards personalizáveis	Usuário	Baixa	Implementado
RF26	Console de eventos fixável na lateral	Usuário	Baixa	Implementado
RF27	Gestão de zonas mortas (áreas de exclusão)	Usuário (Admin), Sistema	Alta	Implementado
RF28	Detecção do Arduino CLI	Usuário, Sistema	Média	Implementado
RF29	Compilação de sketch Arduino pela interface	Usuário, Sistema	Média	Implementado
RF30	Monitor serial na aba Configurações do Arduino	Usuário, Sistema, Arduino	Média	Implementado
RF31	Persistência de preferências da aba Arduino	Sistema	Baixa	Implementado

RF07, RF25 e RF32 foram removidos desta lista por descreverem, respectivamente, um recurso de apoio a desenvolvimento (modo de simulação sem hardware), uma característica de qualidade (responsividade de layout, consolidada em RNF11) e características de portabilidade/instalação (consolidadas em RNF17/RNF18) — não funções acionadas por um ator. Os IDs foram preservados sem renumeração para manter a rastreabilidade histórica. **D1** = divergência entre especificação e implementação atual (o código ainda expõe um caminho de acionamento manual e uma nomenclatura de modos diferente da especificada) — detalhada em `Limitacoes_Conhecidas.md`.

3 Requisitos Não Funcionais

Tabela 2: Requisitos não funcionais do RadarTorres

ID	Requisito	Categoria	Status
RNF01	Redesenho do radar em ~ 150 ms	Desempenho	Atendido
RNF02	Leitura serial não bloqueante	Desempenho	Atendido
RNF03	Degradação graciosa de falhas de hardware/serial	Confiabilidade	Atendido
RNF04	Distância mínima de segurança obrigatória no acionamento	Segurança	Atendido
RNF05	Acionamento é só demonstrativo, nunca armamento real	Segurança	Atendido
RNF06	Senha de usuário nunca em texto puro	Segurança	Atendido
RNF07	Controle de acesso centralizado por perfil	Segurança	Atendido
RNF08	Zonas mortas só editáveis por Administrador	Segurança	Atendido
RNF09	Montagem segura de comandos de processo externo	Segurança	Atendido
RNF10	Arduino CLI nunca baixado automaticamente	Segurança	Atendido
RNF11	Interface responsiva a mudanças de resolução/tamanho	Usabilidade	Atendido
RNF12	Troca de idioma sem reinício	Usabilidade	Atendido
RNF13	Consistência visual entre temas	Usabilidade	Atendido
RNF14	Limite de linhas nos consoles (anti-crescimento de memória)	Desempenho	Atendido
RNF15	Mutação de coleções da UI sempre na thread de UI	Confiabilidade	Atendido
RNF16	Compilação do sketch não bloqueia a interface	Desempenho	Atendido
RNF17	Runtime portátil, sem dependência externa	Portabilidade	Atendido
RNF18	Upgrade preserva configurações do usuário	Manutenibilidade	Atendido
RNF19	Persistência gravável sem privilégio de administrador	Manutenibilidade	Atendido
RNF24	Cobertura de testes automatizados dos componentes críticos	Confiabilidade	Parcial
RNF25	Baud rate configurável	Comunicação	Atendido
RNF26	Codificação e terminador de linha fixos no protocolo	Comunicação	Atendido
RNF27	Plataforma-alvo Windows Desktop	Compatibilidade	Atendido
RNF28	Tolerância a mensagens malformadas do Arduino	Confiabilidade	Atendido

RNF20–RNF23, RNF29 e RNF30 foram removidos desta lista por descreverem decisões arquiteturais internas (persistência substituível, protocolo serial centralizado, MVVM manual, independência de WPF, anticolisão de cards) ou uma funcionalidade ainda não implementada (reconexão automática) — não características de qualidade observáveis do produto. Os IDs foram preservados sem renumeração; o conteúdo real está em `Decisoes_Arquiteturais.md` e `Limitacoes_Conhecidas.md`.

4 Conclusão

Os 29 requisitos funcionais e 24 não funcionais listados cobrem o fluxo central de operação do RadarTorres (comunicação serial, rastreamento, seleção de torre, modos de operação, segurança), os módulos de

suporte administrativo (usuários, auditoria, chamados de ajuda) e as conveniências de interface (idioma, tema, layout do painel). A maioria dos itens está *Implementada* ou *Atendida*; os poucos itens *Parciais* ou *Planejados* (RF16–RF18, RF23, RNF24) têm a lacuna específica registrada nesta tabela e detalhada em *Limitacoes_Conhecidas.md*, sem comprometer o funcionamento do fluxo principal do protótipo.

REFERÊNCIAS

[REFERÊNCIA BIBLIOGRÁFICA A INSERIR]